Методические указания по выполнению заданий на имитацию мобильных сервисов

Переменные и типы данных

В Kotlin переменные бывают изменяемые (var) и только для чтения (val). Основные типы: String (строка), Int (целое), Double (вещественное), Boolean (логический).

Переменные и типы данных

```
val userName: String = "Марина"
var balance: Int = 500
val isActive: Boolean = true
```

val используется, если переменная не меняется.
var для переменных, которые можно менять (например, баланс).

Ввод и вывод данных

Для ввода используется readLine(). Для вывода — println().

Ввод и вывод данных

```
println("Введите имя:")
val name = readLine()
println("Привет, $name!")
```

readLine() читает строку из консоли.
$name — интерполяция строки (вставка переменной в текст).

Условные операторы (if/else) и проверки

if, else позволяют выполнять разные действия при разных условиях.

Условные операторы (if/else) и проверки

```
val age = 17
if (age >= 18) {
    println("Доступ разрешён")
} else {
    println("Доступ запрещён")
}
```

В фигурных скобках — действия для каждого условия.

Оператор выбора (when)

Когда много вариантов выбора — используем when, как расширение if.

Оператор выбора (when)

```
val status = "sent"
when (status) {
    "new" -> println("Новый заказ")
    "sent" -> println("Заказ отправлен")
    "delivered" -> println("Заказ доставлен")
    else -> println("Неизвестный статус")
}
```

when экономичнее, чем несколько if/else подряд.

Массивы и списки

Для хранения двух и более элементов используем массивы (Array) или списки (List).

Массивы и списки

```
val products = listOf("Молоко", "Хлеб", "Масло")
println(products[1])           // выведет: Хлеб
println(products.size)         // размер: 3
```

Нумерация в списках начинается с 0.

listOf — неизменяемый список.

mutableListOf — изменяемый.

Циклы: for, while

Циклы нужны, чтобы обработать много похожих элементов или действий.

Циклы: for, while

```
val basket = listOf("сыр", "чай", "кофе")
for (item in basket) {
    println("В корзине: $item")
}
```

item — отдельный элемент списка на каждой итерации.

Функции

Функция — это блок кода, который можно вызвать по имени и использовать повторно.

Функции

```
fun sum(a: Int, b: Int): Int {
    return a + b
}
val result = sum(2, 3)      // результат будет 5
```

fun объявляет функцию, Int — типы аргументов и результата.

Работа со строками

К строкам применяют методы: длина, поиск символа, изменением регистра, сравнение.

Работа со строками

```
val email = "vasya@mail.ru"
if (email.contains('@')) {
    println("email корректный")
}
println(email.toUpperCase()) // VASYA@MAIL.RU
println("Длина " + email.length)
```

Методы contains, toUpperCase, length для строк.

ООП: Классы и объекты

Класс — шаблон, по которому создают объекты («их собственные» переменные и функции).

ООП: Классы и объекты

```
class User(val name: String, var balance: Int) {
    fun topUp(amount: Int) {
        balance += amount
    }
}
val user = User("Сергей", 200)
user.topUp(500)
println(user.balance)         // 700
```

Конструктор класса задаёт переменные. Методы — действия со значениями.

Сортировка и фильтрация

Для выборки и сортировки элементов списков применяют функции filter, sortedBy, sortedByDescending.

Сортировка и фильтрация

```
val products = listOf(
    Product("Чай", 120),
    Product("Кофе", 340),
    Product("Какао", 290)
)
val expensive = products.filter { it.price > 200 }
val sorted = products.sortedBy { it.price }
```

filter выбирает подходящие по условию элементы,
sortedBy — сортировка по выбранному полю (например, цене).

Коллекции: работа с mutable списками

mutableListOf — коллекция, которую можно изменять: добавлять, удалять элементы.

Коллекции: работа с mutable списками

```
val basket = mutableListOf<String>()
basket.add("Молоко")
basket.add("Сыр")
basket.remove("Молоко")
println(basket)              // [Сыр]
```

add, remove — работа с изменяемым списком.

Генерация кода для проверки условий и бизнес-логики
Валидация регистрации пользователя

Генерация кода для проверки условий и бизнес-логики

```
fun validateRegistration(name: String, age: Int,
email: String, password: String): Boolean {
    if (name.length < 3) {
        println("Имя слишком короткое")
        return false
    }
    if (age < 16) {
        println("Возраст слишком мал")
        return false
    }
    if (!email.contains("@")) {
        println("Некорректный email")
        return false
    }
    if (password.length < 8) {
        println("Слишком короткий пароль")
        return false
    }
    return true
}
```

Функция возвращает true только если все проверки прошли успешно.

Пример сложного ООП: Класс "Order" со статусами

Пример сложного ООП: Класс "Order" со статусами

```
class Product(val name: String, val price: Int)

class Order(val products: List<Product>) {
    var status: String = "new"
    var total: Int = products.sumOf { it.price }
    fun updateStatus(newStatus: String) {
        if (newStatus in listOf("paid", "sent", "delivered", "cancelled"))
{
            status = newStatus
        }
    }
}
```

В классе Order есть поле статуса и функция для его смены.